

2^a Série de Problemas

Trabalho realizado por:

Gustavo Costa | N^o 52808

Ian Frunze | N^o 52867

Rafael Pereira | N^o 52880

Turma: **LEIC24D**

Docentes: **Nuno Leite**

Licenciatura em Engenharia Informática e de Computadores

Algoritmos e Estruturas de Dados (AED)

2024 / 2025 - Semestre de Verão

Conteúdo

1	Introdução	2
2	Problema: Operações entre coleções de pontos no plano	3
2.1	Classe UniquePair	3
2.2	CoreInterface	4
2.3	Implementação 1	4
2.4	Método putList	6
2.5	Implementação 2	6
2.6	Resultados Experimentais	10

1 Introdução

Neste trabalho, desenvolveu-se uma aplicação denominada ProcessPointsCollections, com o objetivo de realizar operações entre coleções de pontos no plano cartesiano. Cada coleção de pontos é representada num ficheiro de texto com extensão .co, onde cada ponto é descrito por um identificador e duas coordenadas (X, Y).

A aplicação permite executar três operações principais sobre os conjuntos de pontos de dois ficheiros: união, interseção e diferença. Estas operações produzem um novo ficheiro .co com os resultados.

A aplicação permite efetuar as seguintes operações entre dois ficheiros de entrada:

- União (union): junta todos os pontos dos dois ficheiros, eliminando repetições. O resultado é um novo ficheiro que contém todos os pontos únicos que aparecem em pelo menos um dos ficheiros de entrada.
- Interseção (intersection): seleciona apenas os pontos que aparecem nos dois ficheiros ao mesmo tempo. O resultado é um ficheiro com os pontos comuns entre os dois conjuntos.
- Diferença (difference): extrai os pontos que estão no primeiro ficheiro mas não aparecem no segundo. O ficheiro resultante contém apenas os pontos exclusivos do primeiro conjunto.

Durante a execução, o programa recebe comandos para carregar os ficheiros, realizar as operações referidas e gerar os ficheiros de saída. Para garantir eficiência nas operações, os pontos são armazenados numa estrutura de dados baseada em hash maps.

Foram implementadas duas versões da aplicação: a primeira recorrendo apenas às estruturas da Kotlin Standard Library, e a segunda utilizando uma estrutura de dados personalizada desenvolvida anteriormente no exercício 4. No final, foi realizada uma avaliação experimental do desempenho de ambas as implementações, recorrendo a ficheiros de teste fornecidos, e os resultados foram apresentados graficamente.

2 Problema: Operações entre coleções de pontos no plano

2.1 Classe UniquePair

```
class UniquePair<T>(var x: T? = null, var y: T? = null) {  
    val full: Boolean  
        get() = x != null && y != null  
  
    fun add(value: T): Boolean{  
        if (full) return false  
  
        if (value == x || value == y) return false  
        else {  
            if (x == null) x = value else y = value  
        }  
        return true  
    }  
  
    fun remove(value: T): Boolean {  
        return when(value) {  
            x -> { x = null; true }  
            y -> { y = null; true }  
            else -> false  
        }  
    }  
  
    fun contains(ele: T): Boolean{  
        return (ele == this.x || ele == this.y)  
    }  
}
```

Código 1: class UniquePair<T>

A classe `UniquePair<T>` representa um par de valores do mesmo tipo, onde só podem existir no máximo dois elementos e estes têm que ser diferentes. Inicialmente, tanto o primeiro valor (x) como o segundo (y) são nulos. A propriedade `full` serve para verificar se já existem dois elementos atribuídos, ou seja, se o par está completo.

O método `add` permite adicionar um novo valor, mas só se ainda houver espaço (ou seja, se o par não estiver completo) e se o valor ainda não existir no par. Se for possível adicionar, o valor é colocado na primeira posição livre. O método devolve `true` se conseguir adicionar e `false` se não for possível.

O método `remove` elimina um valor, se ele existir no par. Se encontrar o valor em x ou y , define esse campo como `null` e devolve `true`. Se o valor não estiver presente, devolve `false`.

O método `contains` verifica se um determinado valor já existe no par, devolvendo `true` se estiver

presente e `false` caso contrário

2.2 CoreInterface

Para garantir uma estrutura clara e modular no desenvolvimento das diferentes implementações, foi definida uma interface chamada `CoreInterface`. Esta interface estabelece o contrato que todas as implementações devem seguir, assegurando que dispõem das funções essenciais para carregar os ficheiros e executar as operações de união, interseção e diferença.

A interface é simples e define os quatro métodos principais, com os seus parâmetros, conforme apresentado na Listagem 2:

```
interface CoreInterface {  
  
    fun load(pathA: String, pathB: String)  
  
    fun union(outName: String)  
  
    fun intersection(outName: String)  
  
    fun difference(outName: String)  
}
```

Código 2: Interface `CoreInterface` que define as funções principais

Desta forma, todas as implementações, incluindo a implementação 1 que utiliza o `HashMap` da biblioteca padrão, garantem consistência e facilitam a manutenção e extensão do código.

2.3 Implementação 1

Implementação 1 da interface principal foi desenvolvida utilizando a estrutura de dados `HashMap` fornecida pela biblioteca padrão do Kotlin. Esta abordagem privilegiou o uso de coleções nativas para gerir o armazenamento e a manipulação dos pontos no plano.

O mapa utilizado associa cada ponto, representado pela classe `Point` com coordenadas `x` e `y` do tipo `Double`, a um par único de etiquetas (`UniquePair<String>`) que indicam os ficheiros onde o ponto aparece (ver Listagem 3):

```
private var points: HashMap<Point, UniquePair<String>> = HashMap()  
  
private data class Point(val x: Double, val y: Double)
```

Código 3: Declaração do mapa e da classe `Point`

A função `load` é responsável por ler dois ficheiros externos contendo pontos no formato esperado, preenchendo o `HashMap` com os pontos e associando-os aos respetivos ficheiros. O processo de leitura

e inserção no mapa é realizado pela função `parseFile`, onde se utiliza o método `computeIfAbsent` para garantir que cada ponto tem um par associado antes de adicionar o identificador do ficheiro (ver Listagem 4):

```
map.computeIfAbsent(point) { UniquePair() }.add(label)
```

Código 4: Uso do método `computeIfAbsent` para adicionar pontos ao mapa

A função `union` escreve num ficheiro todos os pontos carregados, correspondendo à união dos pontos dos dois ficheiros. Antes da escrita, verifica se os dados estão carregados, evitando erros. A escrita é feita pela função `writePointsToFile` que itera pelo mapa e escreve todas as coordenadas. O código da função `union` está apresentado em seguida:

```
override fun union(outName: String) {
    if (!checkData()) return
    writePointsToFile(points, outName)
}
```

Código 5: Função `union` que escreve a união dos pontos

A função `intersection` filtra os pontos que aparecem em ambos os ficheiros, escrevendo apenas estes num ficheiro de saída. Esta filtragem usa a condição `isSimilar` aplicada ao par de ficheiros carregados, garantindo que só os pontos comuns sejam escritos. O código é mostrado abaixo:

```
override fun intersection(outName: String) {
    if (!checkData()) return
    writePointsToFile(points, outName) { it.isSimilar(loadedFileNames) }
}
```

Código 6: Função `intersection` que escreve a interseção dos pontos

A função `difference` produz um ficheiro contendo os pontos que existem só no primeiro ficheiro. Para tal, filtra o mapa para pontos cujo par contém apenas o primeiro ficheiro (o segundo elemento é nulo), utilizando a mesma função de escrita com uma condição apropriada. O código é o seguinte:

```
override fun difference(outName: String) {
    if (!checkData()) return
    val diffCause = UniquePair(loadedFileNames.x, null)
    writePointsToFile(points, outName) { it.isSimilar(diffCause) }
}
```

Código 7: Função `difference` que escreve a diferença dos pontos

Estas funções implementam as operações principais para manipulação dos pontos usando um `HashMap` definido na biblioteca padrão do Kotlin, o que simplifica o armazenamento e a pesquisa dos pontos em função das suas coordenadas.

2.4 Método putList

```
fun <T> putList(key: K, value: T, add: V?.(newValue: T) -> V): V? {  
    val index = (key.hashCode() % capacity).absoluteValue  
    var current = table[index]  
  
    while(current != null){  
        if (current.key == key){  
            val oldValue = current.value  
            current.value = current.value.add(value)  
            return oldValue  
        }  
        current = current.next  
    }  
  
    current = table[index]  
  
    val newNode = HashNode(key, null.add(value))  
    newNode.next = current  
  
    table[index] = newNode  
    size++  
    expand()  
    return null  
}
```

Código 8: Metodo putList

O método `putList` da class `HashMap`, foi criado para lidar com situações em que o valor associado a uma chave no mapa é uma estrutura que pode conter múltiplos elementos, como uma lista. Este método permite adicionar um novo elemento a essa estrutura, mantendo o comportamento do mapa.

A função recebe três parâmetros: uma chave (`key`), um valor a adicionar (`value`) e uma função lambda (`add`) que define como esse valor deve ser adicionado à estrutura existente. Este lambda é uma extensão do tipo lista (que pode ser `null`) utilizado no campo `value` do `HashNode` e deve devolver a estrutura atualizada.

Se já existir um valor para a chave (em caso de colisão), o método aplica a função `add` ao valor atual e atualiza-o. Caso a chave ainda não exista, a função `add` é aplicada sobre `null`, o que permite criar a nova estrutura a partir do zero, e esta é depois inserida no mapa como um novo nó.

2.5 Implementação 2

A classe `CoreImp2` é uma implementação da interface `CoreInterface`, responsável por processar e associar pontos 2D a ficheiros de origem, utilizando uma estrutura de dados `HashMap` personalizada.

Esta estrutura permite associar cada ponto a dois nomes de ficheiros distintos, facilitando operações de comparação entre ficheiros.

A classe mantém várias estruturas auxiliares e variáveis de estado:

```
private var isDataLoaded = false
private var points: HashMap<Point, UniquePair<String>> = HashMap()
private var loadedFileNames: UniquePair<String> = UniquePair()
private data class Point(val x: Double, val y: Double)
```

Código 9: Estado interno da classe CoreImp2

Estas servem para controlar se os dados foram carregados, armazenar os pontos encontrados, registar os nomes dos ficheiros e representar um ponto com coordenadas reais.

Uma função de extensão útil é definida para facilitar a adição de elementos a um `UniquePair`, mesmo que este seja nulo:

```
fun <T> UniquePair<T>?.add(value: T): UniquePair<T> {
    if (this == null) return UniquePair(value)
    this.add(value)
    return this
}
```

Código 10: Extensão add para UniquePair

Esta função permite adicionar elementos de forma segura, evitando exceções em casos onde o par ainda não foi inicializado.

O método principal de carregamento de dados é `load`, que lê dois ficheiros, extrai os pontos válidos e associa-os ao respetivo nome de ficheiro:

```
override fun load(pathA: String, pathB: String) {
    println("${BLUE}Code: Loading $pathA and $pathB$RESET")
    try {
        val fileA = File(pathA).name
        val fileB = File(pathB).name
        points = loadPointsMap(pathA to fileA, pathB to fileB)
        loadedFileNames.x = fileA
        loadedFileNames.y = fileB
        isDataLoaded = true
        println("${GREEN}Core: Data loaded successfully. $RESET")
    } catch (e: Exception) {
        points = HashMap()
        loadedFileNames = UniquePair()
        isDataLoaded = false
        println("${RED}Core error: An error occurred: ${e.message}$RESET")
    }
}
```



```
}
```

Código 11: Método load

Após os dados estarem carregados, é possível realizar operações como união, interseção e diferença entre os pontos dos dois ficheiros. Os métodos `union`, `intersection` e `difference` produzem ficheiros com os resultados destas operações:

```
override fun intersection(outName: String) {
    if (!checkData()) return
    writePointsToFile(points, outName) { it.isSimilar(loadedFileNames) }
    println("${BLUE}Code: Intersection written to $outName$RESET")
}
```

Código 12: Exemplo do método intersection

Cada um destes métodos utiliza a função auxiliar `writePointsToFile`, que recebe um predicado opcional para filtrar os pontos a escrever.

O método `loadPointsMap` agrega os pontos a partir de múltiplos ficheiros, e delega a leitura linha a linha a `parseFile`:

```
private fun loadPointsMap(vararg files: Pair<String, String>): HashMap<Point,
    UniquePair<String>> {
    val map = HashMap<Point, UniquePair<String>>()
    for ((path, label) in files) {
        parseFile(path, map, label)
    }
    return map
}
```

Código 13: Função loadPointsMap

A função `parseFile` é responsável por validar e extrair pontos com coordenadas válidas:

```
private fun parseFile(file: String, map: HashMap<Point, UniquePair<String>>, label:
    String) {
    try {
        File(file).forEachLine { line ->
            if (line.startsWith("v")) {
                val parts = line.trim().split(" ")
                if (parts.size >= 4) {
                    val x = parts[2].toDoubleOrNull()
                    val y = parts[3].toDoubleOrNull()
                    if (x == null || y == null) {
                        println("${YELLOW}Core warning: Invalid line in file $file
-> $line$RESET")
                        throw IllegalArgumentException("Invalid coordinates")
                    }
                }
            }
        }
    }
```

```

        val point = Point(x, y)
        map.putList(point, label) { add(label) }
    }
}
} catch (e: Exception) {
    println("${RED}Core error: Failed to parse file $file due to ${e.message}
$RESET")
}
}

```

Código 14: Função `parseFile`

Utiliza o método `putList` para inserir no `map`.

A escrita dos pontos num ficheiro de saída é feita por `writePointsToFile`, que pode receber uma função de condição:

```

private fun writePointsToFile(
    points: HashMap<Point, UniquePair<String>>,
    filename: String,
    condition: (point: UniquePair<String>) -> Boolean = { true }
) {
    try {
        File(filename).bufferedWriter().use { writer ->
            for (ele in points) {
                val files = ele.value
                val point = ele.key
                if (condition(files)) {
                    writer.write("${point.x.toLong()} ${point.y.toLong()}\n")
                }
            }
        }
    } catch (e: Exception) {
        println("${RED}Core error: Failed to write to file $filename due to ${e.
message}$RESET")
    }
}

```

Código 15: Função `writePointsToFile`

Por fim, a função `checkData` garante que os dados foram previamente carregados antes de executar qualquer operação de processamento:

```

private fun checkData(): Boolean {
    if (!isDataLoaded) {
        println("${RED}Core error: Data not loaded. Use load <fileA> <fileB> first.
$RESET")
    }
}

```

```
        return false
    }
    return true
}
```

Código 16: Função checkData

2.6 Resultados Experimentais

Para avaliar o desempenho da aplicação desenvolvida, foi criado um sistema automatizado de testes capaz de executar operações sobre ficheiros gerados dinamicamente com diferentes quantidades de pontos. Esse sistema baseia-se na leitura de um ficheiro de comandos, que simula a execução real por parte de um utilizador.

Foram criados ficheiros de teste com nomes do tipo `teste10`, `teste100`, até `teste10000000`, representando o número de pontos contidos nos ficheiros gerados. Cada teste cria dois ficheiros de pontos com a quantidade especificada e executa, de forma sequencial, as operações fundamentais suportadas pelo sistema: `load`, `union`, `intersection` e `difference`. Como exemplo está representada abaixo a implementação do ficheiro `test10.ppc`.

```
// Test file script

// Create a test file with 10 positive points between 0 and 100000
mtf Tests/A.co 10 0 100000 0 100000

// Create a second test file with 10 positive points between 0 and 100000
mtf Tests/B.co 10 0 100000 0 100000

// Move into the Tests directory
cd Tests

// Measure the time taken to load the files A and B
time load A.co B.co

// Measure the time taken to perform the union operation on the two files
time u result.co

// Measure the time taken to perform the intersection operation on the two files
time i result.co

// Measure the time taken to perform the difference operation on the two files
time d result.co
```

Código 17: Ficheiro test10.ppc

Durante a execução, o tempo necessário para cada operação foi medido e registado, permitindo uma análise detalhada do desempenho da aplicação à medida que a quantidade de dados aumenta. Esta abordagem possibilitou a comparação objetiva entre diferentes implementações do sistema, nomeadamente a CoreImp1 e a CoreImp2, com foco na eficiência de cada uma nas diversas operações.

De seguida, apresentam-se os gráficos com os tempos de execução das funções **load**, **union**, **intersection** e **difference** para ambas as implementações. Estes gráficos foram gerados com base em ficheiros de teste contendo entre 10 e 10.000.000 pontos, nos quais se mediu o tempo necessário para cada operação.

Note que, para as funções **union**, **intersection** e **difference**, a complexidade do pior caso é $O(n + m)$ em todas.

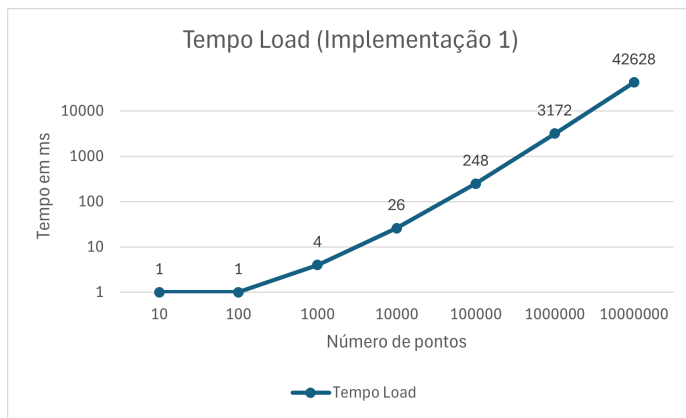


Figura 1: Load – Implementação 1

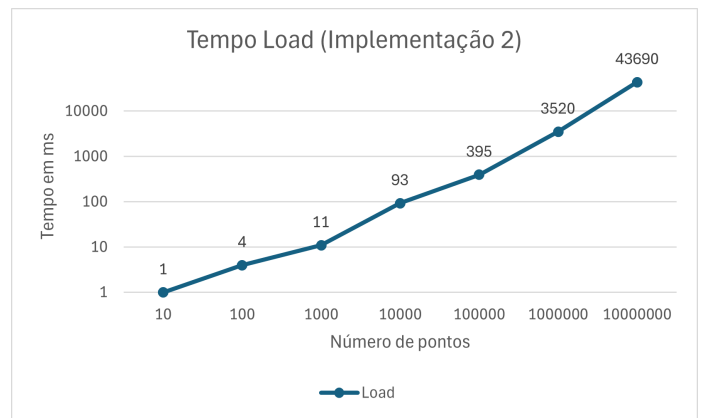


Figura 2: Load – Implementação 2

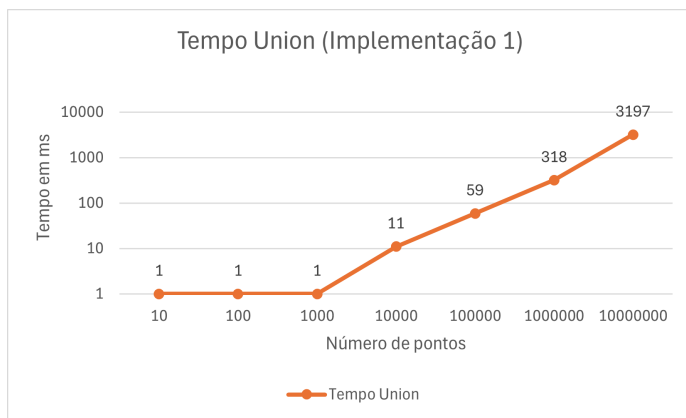


Figura 3: Union – Implementação 1

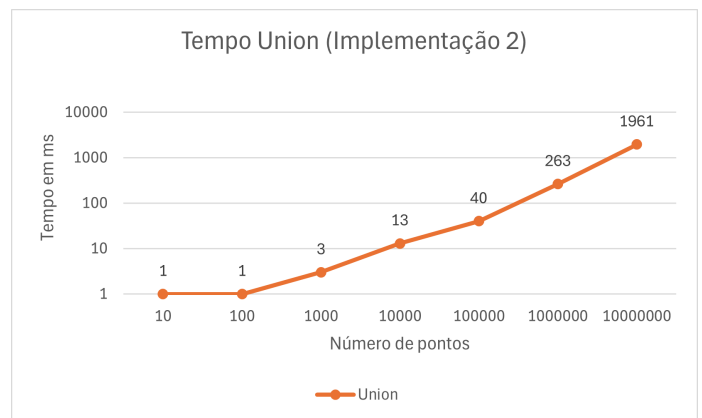


Figura 4: Union – Implementação 2

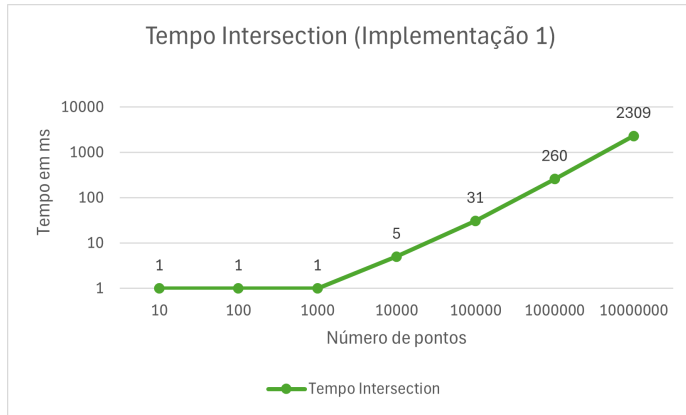


Figura 5: Intersection – Implementação 1

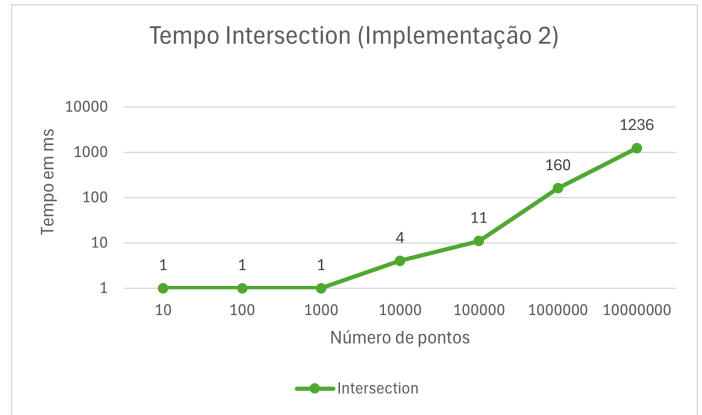


Figura 6: Intersection – Implementação 2

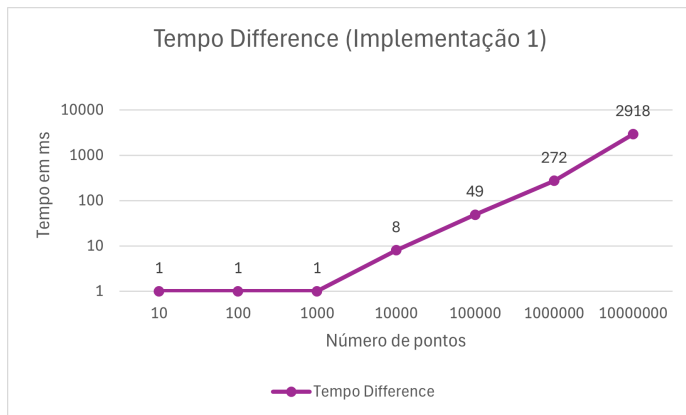


Figura 7: Difference – Implementação 1

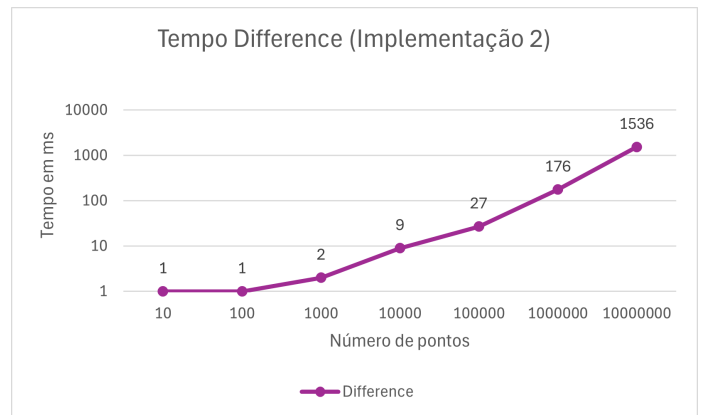


Figura 8: Difference – Implementação 2

No geral, todas as funções apresentam um desempenho melhor na Implementação 2 em comparação com a Implementação 1, com exceção da função `load`, que é ligeiramente mais lenta na Implementação 2. Este comportamento poderá estar relacionado com a forma como a estrutura de dados é inicializada e preenchida nas duas versões, bem como com as otimizações internas feitas na manipulação dos pontos e pares únicos.

As funções de `union`, `intersection` e `difference` beneficiam da estrutura mais eficiente da Implementação 2, refletindo-se nos tempos menores medidos. Esta análise reforça a importância de escolher a estrutura de dados adequada para otimizar o desempenho em operações de conjunto sobre coleções de pontos no plano.